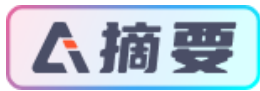


FreeRTOS任务通知 基于STM32



本文围绕FreeRTOS的任务通知功能展开，介绍了其自V8.2.0版本开始提供，可替代二值信号量、计数信号量等，阐述了运作机制，详细讲解多个函数接口，还通过实验展示其代替消息队列、二值信号量和计数信号量的应用及现象。

文章目录

[一、任务通知简介](#)

[二、任务通知的运作机制](#)

[三、任务通知的函数接口讲解](#)

[1. xTaskGenericNotify\(\)](#)

[2.xTaskNotifyGive\(\)](#)

[3.vTaskNotifyGiveFromISR\(\)](#)

[4.xTaskNotify\(\)](#)

[5. xTaskNotifyFromISR\(\)](#)

[6.xTaskNotifyAndQuery\(\)](#)

[7.xTaskNotifyAndQueryFromISR\(\)](#)

[8.ulTaskNotifyTake\(\)](#)

[9.xTaskNotifyWait\(\)](#)

[四、任务通知实验](#)

[1 任务通知代替消息队列](#)

[2 任务通知代替二值信号量](#)

[3 任务通知代替计数信号量](#)

[五、实验现象](#)

一、任务通知简介

FreeRTOS 从 V8.2.0 版本开始提供任务通知这个功能，每个任务都有一个 32 位的通知值，在大多数情况下，任务通知可以替代二值信号量、计数信号量、事件组，也可以替代长度为 1 的队列（可以保存一个 32位整数或指针值）。

相对于以前使用 FreeRTOS 内核通信的资源，必须创建队列、二进制信号量、计数信号量或事件组的情况，使用任务通知显然更灵活。

按照 FreeRTOS 官方的说法，使用任务通知比通过信号量等 ICP 通信方式解除阻塞的任务要快 45%，并且更加省 RAM 内存空间（使用 GCC 编译器，-o2 优化级别），任务通知的使用无需创建队列。想要使用任务通知，必须将 FreeRTOSConfig.h 中的宏定义 configUSE_TASK_NOTIFICATIONS 设置为 1，其实 FreeRTOS 默认是为 1 的，所以任务通知是默认使能的。

FreeRTOS 提供以下几种方式发送通知给任务：

发送通知给任务，如果有通知未读，不覆盖通知值。

发送通知给任务，直接覆盖通知值。

发送通知给任务，设置通知值的一个或者多个位，可以当做事件组来使用。

发送通知给任务，递增通知值，可以当做计数信号量使用。通过对以上任务通知方式的合理使用，可以在一定场合下替代 FreeRTOS 的信号量，队列、事件组等。

当然，凡是都有利弊，不然的话 FreeRTOS 还要内核的 IPC 通信机制干嘛，消息通知 虽然处理更快，RAM 开销更小，但也有以下限制：

只能有一个任务接收通知消息，因为必须指定接收通知的任务。。

只有等待通知的任务可以被阻塞，发送通知的任务，在任何情况下都不会因为发送失败而进入阻塞态。

二、任务通知的运作机制

顾名思义，任务通知是属于任务中附带的资源，所以在任务被创建的时候，任务通知 也被初始化的，而在分析队列和信号量的章节中，我们知道在使用队列、信号量前，必须 先创建队列和信号量，目的是为了创建队列数据结构。比如使用 xQueueCreate()函数创建 队列，用 xSemaphoreCreateBinary()函数创建二值信号量等等。再来看任务通知，由于任务通知的数据结构包含在任务控制块中，只要任务存在，任务通知数据结构就已经创建完毕，可以直接使用，所以使用的时候很是方便。任务通知可以在任务中向指定任务发送通知，也可以在中断中向指定任务发送通知，FreeRTOS 的每个任务都有一个 32 位的通知值，任务控制块中的成员变量 ulNotifiedValue 就是这个通知值。只有在任务中可以等待通知，而不允许在中断中等待通知。如果任务在等待的通知暂时无效，任务会根据用户指定的阻塞超时时间进入阻塞状态，我们可以将等待通知的任务看作是消费者；其它任务和中断可以向等待通知的任务发送通知，发送通知的任务和中断服务函数可以看作是生产者，当其他任务或者中断向这个任务发送任务通知，任务获得通知以后，该任务就会从阻塞态中解除，这与 FreeRTOS 中内核的其他通信机制一致。

三、任务通知的函数接口讲解

1. xTaskGenericNotify()

我们先看一下发送通知 API 函数。这类函数比较多，有 6 个。但仔细分析会发现它们 只能完成 3 种操作，每种操作有两个 API 函数，分别为带中断保护版本和不带中断保护版本。FreeRTOS 将 API 细分为带中断保护版本和不带中断保护版本是为了节省中断服务程序处理时间，提升性能。通过前面通信机制的学习，相信大家都了解了 FreeRTOS 的风格，这里的任务通知发送函数也是利用宏定义来进行扩展的，所有的函数都是一个宏定义，在任务中发送任务通知的函数均是调用 xTaskGenericNotify()函数进行发送通知，xTaskGenericNotify()函数是一个通用的任务通知发送函数，在任务中发送通知的 API 函数，如 xTaskNotifyGive()、xTaskNotify()，xTaskNotifyAndQuery()，都是以 xTaskGenericNotify()为原型的，只不过指定的发送方式不同而已。

2.xTaskNotifyGive()

xTaskNotifyGive()是一个宏，宏展开是调用函数 xTaskNotify((xTaskToNotify), (0), eIncrement)，即向一个任务发送通知，并将对方的任务通知值加 1。该函数可以作为二值 信号量和计数信号量的一种轻量型的实现，速度更快，在这种情况下对象任务在等待任务通知的时候应该是使用函数 ulTaskNotifyTake() 而不是 xTaskNotifyWait()。xTaskNotifyGive() 不能在中断里面使用，而是使用具有中断保护功能的 vTaskNotifyGiveFromISR()来代替。

xTaskNotifyGive()函数说明

函数原型	<pre>#define xTaskNotifyGive(xTaskToNotify) xTaskGenericNotify((xTaskToNotify), (0), eIncrement, NULL)</pre>	
功能	用于在任务中向指定任务发送任务通知，并更新对方的任务通知值（加 1 操作）。	
参数	xTaskToNotify	接收通知的任务句柄，并让其自身的任务通知值加 1。
返回值	总是返回 pdPASS。	

CSDN @不秃也很强

xTaskNotifyGive()函数应用举例

```

1. static void prvTask1( void *pvParameters );
2. static void prvTask2( void *pvParameters );
3.
4. /*定义任务句柄 */
5. static TaskHandle_t xTask1 = NULL, xTask2 = NULL;
6.
7. /* 主函数:创建两个任务，然后开始任务调度 */
8. void main( void )
9. {
10. xTaskCreate(prvTask1, "Task1", 200, NULL, tskIDLE_PRIORITY, &xTask1);
11. xTaskCreate(prvTask2, "Task2", 200, NULL, tskIDLE_PRIORITY, &xTask2);
12. vTaskStartScheduler();
13. }
14. /*-----*/
15.
16. static void prvTask1( void *pvParameters )
17. {
18. for ( ;; ) {
19. /* 向 prvTask2()发送一个任务通知，让其退出阻塞状态 */
20. xTaskNotifyGive( xTask2 );
21.
22. /* 阻塞在 prvTask2()的任务通知上
23. 如果没有收到通知，则一直等待*/
24. ulTaskNotifyTake( pdTRUE, portMAX_DELAY );
25. }
26. }
27. /*-----*/
28.
29. static void prvTask2( void *pvParameters )
30. {
31. for ( ;; ) {
32. /* 阻塞在 prvTask1()的任务通知上
33. 如果没有收到通知，则一直等待*/
34. ulTaskNotifyTake( pdTRUE, portMAX_DELAY );
35.
36. /* 向 prvTask1()发送一个任务通知，让其退出阻塞状态 */
37. xTaskNotifyGive( xTask1 );
38. }
39. }
```



3.vTaskNotifyGiveFromISR()

vTaskNotifyGiveFromISR()是 vTaskNotifyGive()的中断保护版本。用于在中断中向指定 任务发送任务通知，并更新对方的任务通知值（加 1 操作），在某些场景中可以替代信号 量操作，因为这两个通知都是不带有通知值的。

vTaskNotifyGiveFromISR()函数说明

函数原型	void vTaskNotifyGiveFromISR(TaskHandle_t xTaskToNotify, BaseType_t *pxHigherPriorityTaskWoken);	
功能	用于在中断中向一个任务发送任务通知，并更新对方的任务通知值（加 1 操作）。	
参数	xTaskToNotify	接收通知的任务句柄，并让其自身的任务通知值加 1。
	pxHigherPriorityTaskWoken	*pxHigherPriorityTaskWoken 在使用之前必须先初始化为 pdFALSE。当调用该函数发送一个任务通知时，目标任务接收到通知后将从阻塞态变为就绪态，并且如果其优先级比当前运行的任务的优先级高，那么 *pxHigherPriorityTaskWoken 会被设置为 pdTRUE，然后在中断退出前执行一次上下文切换，去执行刚刚被唤醒的中断优先级较高的任务。pxHigherPriorityTaskWoken 是一个可选的参数可以设置为 NULL。
返回值	无。	

CSDN @不秃也很强

从上面的函数说明我们大概知道 vTaskNotifyGiveFromISR()函数作用，每次调用该函数都会增加任务的通知值，任务通过接收函数返回值是否大于零，判断是否获取到了通知，任务通知值初始化为 0，（如果与信号量做对比）则对应为信号量无效。当中断调用 vTaskNotifyGiveFromISR()通知函数给任务的时候，任务的通知值增加，使其大于零，使其表示的通知值变为有效，任务获取有效的通知值将会被恢复。

vTaskNotifyGiveFromISR()函数应用举例

```

1. static TaskHandle_t xTaskToNotify = NULL;
2.
3. /* 外设驱动的数据传输函数 */
4. void StartTransmission( uint8_t *pcData, size_t xDataLength )
5. {
6. /* 在这个时候，xTaskToNotify 应为 NULL，因为发送并没有进行。
7. 如果有必要，对外设的访问可以用互斥量来保护*/
8. configASSERT( xTaskToNotify == NULL );
9.
10. /* 获取调用函数 StartTransmission() 的任务的句柄 */
11. xTaskToNotify = xTaskGetCurrentTaskHandle();
12.
13. /* 开始传输，当数据传输完成时产生一个中断 */
14. vStartTransmit( pcData, xDataLength );
15. }
16. /*-----*/
17. /* 数据传输完成中断 */
18. void vTransmitEndISR( void )
19. {
20. BaseType_t xHigherPriorityTaskWoken = pdFALSE;
21.
22. /* 这个时候不应该为 NULL，因为数据传输已经开始 */
23. configASSERT( xTaskToNotify != NULL );
24.
25. /* 通知任务传输已经完成 */
26. vTaskNotifyGiveFromISR( xTaskToNotify, &xHigherPriorityTaskWoken );
27.
28. /* 传输已经完成，所以没有任务需要通知 */
29. xTaskToNotify = NULL;
30.
31. /* 如果为 pdTRUE，则进行一次上下文切换 */
32. portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
33. }
34. /*-----*/
35. /* 任务：启动数据传输，然后进入阻塞态，直到数据传输完成 */
36. void vFunctionCalledFromATask( uint8_t ucDataToTransmit,
37. size_t xDataLength )

```

```
38. {
39.   uint32_t ulNotificationValue;
40.   const TickType_t xMaxBlockTime = pdMS_TO_TICKS( 200 );
41.
42.   /* 调用上面的函数 StartTransmission()启动传输 */
43.   StartTransmission( ucDataToTransmit, xDataLength );
44.
45.   /* 等待传输完成 */
46.   ulNotificationValue = ulTaskNotifyTake( pdFALSE, xMaxBlockTime );
47.
48.   /* 当传输完成时，会产生一个中断
49.   在中断服务函数中调用 vTaskNotifyGiveFromISR()向启动数据
50.   传输的任务发送一个任务通知，并将对象任务的任务通知值加 1
51.   任务通知值在任务创建的时候是初始化为 0 的，当接收到任务后就变成 1 */
52.   if ( ulNotificationValue == 1 ) {
53.     /* 传输按预期完成 */
54.   } else {
55.     /* 调用函数 ulTaskNotifyTake()超时 */
56.   }
57. }
```



4.xTaskNotify()

FreeRTOS 每个任务都有一个 32 位的变量用于实现任务通知，在任务创建的时候初始化为 0。这个 32 位的通知值在任务控制块 TCB 里面定义，具体见代码清单 22-6。xTaskNotify()用于在任务中直接向另外一个任务发送一个事件，接收到该任务通知的任务有可能解锁。如果你想使用任务通知来实现二值信号量和计数信号量，那么应该使用更加简单的函数 xTaskNotifyGive()，而不是使用 xTaskNotify()，xTaskNotify()函数在发送任务通知的时候会指定一个通知值，并且用户可以指定通知值发送的方式。注意：该函数不能在中断里面使用，而是使用具体中断保护功能的版本函数 xTaskNotifyFromISR()。

xTaskNotify()函数说明

函数原型	BaseType_t xTaskNotify(TaskHandle_t xTaskToNotify, uint32_t ulValue, eNotifyAction eAction);	
功能	向指定的任务发送一个任务通知，带有通知值并且用户可以指定通知值的发送方式。	
参数	xTaskToNotify	需要接收通知的任务句柄。
	ulValue	用于更新接收任务通知的任务通知值，具体如何更新由形参 eAction 决定。
	eAction	任务通知值更新方式，具体见表格 22-4。
返回值	参数 eAction 为 eSetValueWithoutOverwrite 时，如果被通知任务还没取走上一个通知，又接收到了一个通知，则这次通知值未能更新并返回 pdFALSE，而其他情况均返回 pdPASS。	

CSDN @不秃也很强

xTaskNotify()函数应用举例

```
1. /* 设置任务 xTask1Handle 的任务通知值的位 8 为 1*/
2. xTaskNotify( xTask1Handle, ( 1UL << 8UL ), eSetBits );
3.
4. /* 向任务 xTask2Handle 发送一个任务通知
5. 有可能会解除该任务的阻塞状态，但是并不会更新该任务自身的任务通知值 */
6. xTaskNotify( xTask2Handle, 0, eNoAction );
7.
8.
9. /* 向任务 xTask3Handle 发送一个任务通知
10. 并把该任务自身的任务通知值更新为 0x50
11. 即使该任务的上一次的任务通知都没有读取的情况下
12. 即覆盖写 */
13. xTaskNotify( xTask3Handle, 0x50, eSetValueWithOverwrite );
```



```
14.
15. /* 向任务 xTask4Handle 发送一个任务通知
16. 并把该任务自身的任务通知值更新为 0xffff
17. 但是并不会覆盖该任务之前接收到的任务通知值*/
18. if(xTaskNotify(xTask4Handle, 0xffff, eSetValueWithoutOverwrite)==pdPASS )
19. {
20. /* 任务 xTask4Handle 的任务通知值已经更新 */
21. } else
22. {
23. /* 任务 xTask4Handle 的任务通知值没有更新
24. 即上一次的通知值还没有被取走*/
25. }
```



5. xTaskNotifyFromISR()

xTaskNotifyFromISR()是 xTaskNotify()的中断保护版本，真正起作用的函数是中断发送 任务通知通用函数 xTaskGenericNotifyFromISR()，而 xTaskNotifyFromISR()是一个宏定义，用于在中断中向指定的任务发送一个任务通知，该任务通知是带有 通知值并且用户可以指定通知的发送方式，不返回上一个任务在的通知值。

xTaskNotifyFromISR()函数说明

函数原型	BaseType_t xTaskNotifyFromISR(TaskHandle_t xTaskToNotify, uint32_t ulValue, eNotifyAction eAction, BaseType_t *pxHigherPriorityTaskWoken);	
功能	在中断中向指定的任务发送一个任务通知。	
参数	xTaskToNotify	指定接收通知的任务句柄。
	ulValue	用于更新接收任务通知的任务通知值，具体如何更新由形参 eAction 决定。
	eAction	任务通知值的状态，具体见表格 22-4。
	pxHigherPriorityTaskWoken	*pxHigherPriorityTaskWoken 在使用之前必须先初始化为 pdFALSE。当调用该函数发送一个任务通知时，目标任务接收到通知后将从阻塞态变为就绪态，并且如果其优先级比当前运行的任务的优先级高，那么 *pxHigherPriorityTaskWoken 会被设置为 pdTRUE，然后在中断退出前执行一次上下文切换，去执行刚刚被唤醒的中断优先级较高的任务。pxHigherPriorityTaskWoken 是一个可选的参数可以设置为 NULL。
返回值	参数 eAction 为 eSetValueWithoutOverwrite 时，如果被通知任务还没取走上一个通知，又接收到了一个通知，则这次通知值未能更新并返回 pdFALSE，其他情况均返回 pdPASS。	

CSDN @不秃也很强

中断中发送任务通知通用函数 xTaskGenericNotifyFromISR()

xTaskGenericNotifyFromISR() 是一个在中断中发送任务通知的通用函数， xTaskNotifyFromISR()、xTaskNotifyAndQueryFromISR()等函数都是以其为基础，采用宏定义的方式实现。

xTaskNotifyFromISR()使用实例

```
1. /* 中断：向一个任务发送任务通知，并根据不同的中断将目标任务的
2. 任务通知值的相应位置 1 */
3. void vANInterruptHandler( void )
4. {
```

```

5. BaseType_t xHigherPriorityTaskWoken;
6. uint32_t ulStatusRegister;
7.
8. /* 读取中断状态寄存器，判断到来的是哪个中断
9. 这里假设了 Rx、Tx 和 buffer overrun 三个中断 */
10. ulStatusRegister = ulReadPeripheralInterruptStatus();
11.
12. /* 清除中断标志位 */
13. vClearPeripheralInterruptStatus( ulStatusRegister );
14.
15. /* xHigherPriorityTaskWoken 在使用之前必须初始化为 pdFALSE
16. 如果调用函数 xTaskNotifyFromISR() 解锁了解锁了接收该通知的任务
17. 而且该任务的优先级比当前运行的任务的优先级高，那么
18. xHigherPriorityTaskWoken 就会自动的被设置为 pdTRUE*/
19. xHigherPriorityTaskWoken = pdFALSE;
20.
21. /* 向任务 xHandlingTask 发送任务通知，并将其任务通知值
22. 与 ulStatusRegister 的值相或，这样可以不改变任务通知其它位的值*/
23. xTaskNotifyFromISR( xHandlingTask,
24. ulStatusRegister,
25. eSetBits,
26. &xHigherPriorityTaskWoken );
27.
28. /* 如果 xHigherPriorityTaskWoken 的值为 pdTRUE 则执行一次上下文切换*/
29. portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
30. }
31. /* ----- */
32.
33.
34. /* 任务：等待任务通知，然后处理相关的事情 */
35. void vHandlingTask( void *pvParameters )
36. {
37. uint32_t ulInterruptStatus;
38.
39. for ( ;; ) {
40. /* 等待任务通知，无限期阻塞（没有超时，所以没有必要检查函数返回值）*/
41. xTaskNotifyWait( 0x00, /* 在进入的时候不清除通知值的任何位 */
42. ULONG_MAX, /* 在退出的时候复位通知值为 0 */
43. &ulNotifiedValue, /* 任务通知值传递到变量
44. ulNotifiedValue 中 */
45. portMAX_DELAY ); /* 无限期等待 */
46.
47. /* 根据任务通知值里面的各个位的值处理事情 */
48. if ( ( ulInterruptStatus & 0x01 ) != 0x00 ) {
49. /* Rx 中断 */
50. prvProcessRxInterrupt();
51. }
52.
53. if ( ( ulInterruptStatus & 0x02 ) != 0x00 ) {
54. /* Tx 中断 */
55. prvProcessTxInterrupt();
56. }
57.
58. if ( ( ulInterruptStatus & 0x04 ) != 0x00 ) {
59. /* 缓冲区溢出中断 */
60. prvClearBufferOverrun();
61. }
62. }
63. }

```



6.xTaskNotifyAndQuery()

xTaskNotifyAndQuery()与 xTaskNotify()很像，都是调用通用的任务通知发送函数 xTaskGenericNotify() 来实现通知的发送，不同的是多了一个附加的参数 pulPreviousNotifyValue 用于回传接收任务的上一个通知值，xTaskNotifyAndQuery()函数不能用在中断中，而是必须使用带中断保护功能的 xTaskNotifyAndQuery()FromISR 来代替。

xTaskNotifyAndQuery()函数说明

函数原型	BaseType_t xTaskNotifyAndQuery(TaskHandle_t xTaskToNotify, uint32_t ulValue, eNotifyAction eAction, uint32_t *pulPreviousNotifyValue);	
功能	向指定的任务发送一个任务通知，并返回对象任务的上一个通知值。	
参数	xTaskToNotify	需要接收通知的任务句柄。
	ulValue	用于更新接收任务通知的任务通知值，具体如何更新由形参 eAction 决定。
	eAction	任务通知值更新方式，具体见表格 22-4。
	pulPreviousNotifyValue	对象任务的上一个任务通知值，如果为 NULL，则不需要回传，这个时候就等价于函数 xTaskNotify()。
返回值	参数 eAction 为 eSetValueWithoutOverwrite 时，如果被通知任务还没取走上一个通知，又接收到了一个通知，则这次通知值未能更新并返回 pdFALSE，其他情况均返回 pdPASS。 CSDN @不秃也很强	

7.xTaskNotifyAndQueryFromISR()

xTaskNotifyAndQueryFromISR()是 xTaskNotifyAndQuery ()的中断版本，用于向指定的 任务发送一个任务通知，并返回对象任务的上一个通知值，该函数也是一个宏定义，真正 实现发送通知的是 xTaskGenericNotifyFromISR()。

xTaskNotifyAndQueryFromISR()函数说明

函数原型	BaseType_t xTaskNotifyAndQueryFromISR(TaskHandle_t xTaskToNotify, uint32_t ulValue, eNotifyAction eAction, uint32_t *pulPreviousNotifyValue, BaseType_t *pxHigherPriorityTaskWoken);	
功能	在中断中向指定的任务发送一个任务通知，并返回对象任务的上一个通知值。	
参数	xTaskToNotify	需要接收通知的任务句柄。
	ulValue	用于更新接收任务通知的任务通知值，具体如何更新由形参 eAction 决定。
	eAction	任务通知值的状态，具体见表格 22-4。
	pulPreviousNotifyValue	对象任务的上一个任务通知值。如果为 NULL，则不需要回传。
	pxHigherPriorityTaskWoken	*pxHigherPriorityTaskWoken 在使用之前必须先初始化为 pdFALSE。当调用该函数发送一个任务通知时，目标任务接收到通知后将从阻塞态变为就绪态，并且如果其优先级比当前运行的任务的优先级高，那么 *pxHigherPriorityTaskWoken 会被设置为 pdTRUE，然后在中断退出前执行一次上下文切换，去执行刚刚被唤醒的中断优先级较高的任务。pxHigherPriorityTaskWoken 是一个可选的参数可以设置为 NULL。
返回值	参数 eAction 为 eSetValueWithoutOverwrite 时，如果被通知任务还没取走上一个通知，又接收到了一个通知，则这次通知值未能更新并返回 pdFALSE，其他情况均返回 pdPASS。	

CSDN @不秃也很强

获取任务通知函数

既然 FreeRTOS 中发送任务的函数有那么多，那么任务怎么获取到通知呢？我们说了，任务通知在某些场景可以替代信号量、消息队列、事件等。获取任务通知函数只能用在任务中，没有带中断保护版本，因此只有两个 API 函数：ulTaskNotifyTake() 和 xTaskNotifyWait()。前者是为代替二值信号量和计数信号量而专门设计的，它和发送通知 API 函数 xTaskNotifyGive()、vTaskNotifyGiveFromISR() 配合使用；后者是全功能版的等待通知，可以根据不同的参数实现轻量级二值信号量、计数信号量、事件组和长度为 1 的队列。

所有的获取任务通知 API 函数都带有指定阻塞超时时间参数，当任务因为等待通知而进入阻塞时，用来指定任务的阻塞时间，这些超时机制与 FreeRTOS 的消息队列、信号量、事件等的超时机制一致。

8.ulTaskNotifyTake()

ulTaskNotifyTake() 作为二值信号量和计数信号量的一种轻量级实现，速度更快。如果 FreeRTOS 中使用函数 xSemaphoreTake() 来获取信号量，这个时候则可以试试使用函数 ulTaskNotifyTake() 来代替。对于这个函数，任务通知值为 0，对应信号量无效，如果任务设置了阻塞等待，任务被阻塞挂起。当其他任务或中断发送了通知值使其不为 0 后，通知变为有效，等待通知的任务将获取到通知，并且在退出时候根据用户传递的第一个参数 xClearCountOnExit 选择清零通知值或者执行减一操作。xTaskNotifyTake() 在退出的时候处理任务的通知值的时候有两种方法，一种是在函数退出时将通知值清零，这种方法适用于实现二值信号量；另外一种是在函数退出时将通知值减 1，这种方法适用于实现计数信号量。

当一个任务使用其自身的任务通知值作为二值信号量或者计数信号量时，其他任务应该使用函数 xTaskNotifyGive() 或者 xTaskNotify((xTaskToNotify), (0), eIncrement) 来向其发送信号量。如果是在中断中，则应该使用他们的中断版本函数。

ulTaskNotifyTake()函数说明

函数原型	uint32_t ulTaskNotifyTake(BaseType_t xClearCountOnExit, TickType_t xTicksToWait);	
功能	用于获取一个任务通知，获取二值信号量、计数信号量类型的任务通知。	
参数	xClearCountOnExit	设置为 pdFALSE 时，函数 xTaskNotifyTake()退出前，将任务的通知值减 1，可以用来实现计数信号量；设置为 pdTRUE 时，函数 xTaskNotifyTake()退出前，将任务通知值清零，可以用来实现二值信号量。
	xTicksToWait	超时时间，单位为系统节拍周期。宏 pdMS_TO_TICKS 用于将毫秒转化为系统节拍数。
返回值	返回任务的当前通知值，在其减 1 或者清 0 之前。	

CSDN @不秃也很强

ulTaskNotifyTake()函数应用举例

```

1.  /* 中断服务程序：向一个任务发送任务通知 */
2.  void vANInterruptHandler( void )
3.  {
4.      BaseType_t xHigherPriorityTaskWoken;
5.
6.      /* 清除中断 */
7.      prvClearInterruptSource();
8.
9.      /* xHigherPriorityTaskWoken 在使用之前必须设置为 pdFALSE
10.     如果调用 vTaskNotifyGiveFromISR()会解除 vHandlingTask 任务的阻塞状态，
11.     并且 vHandlingTask 任务的优先级高于当前处于运行状态的任务，
12.     则 xHigherPriorityTaskWoken 将会自动被设置为 pdTRUE */
13.     xHigherPriorityTaskWoken = pdFALSE;
14.
15.     /* 发送任务通知，并解锁阻塞在该任务通知下的任务 */
16.     vTaskNotifyGiveFromISR( xHandlingTask, &xHigherPriorityTaskWoken );
17.
18.     /* 如果被解锁的任务优先级比当前运行的任务的优先级高
19.     则在中断退出前执行一次上下文切换，在中断退出后去执行
20.     刚刚被唤醒的优先级更高的任务*/
21.     portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
22. }
23. /*-----*/
24. /* 任务：阻塞在一个任务通知上 */
25. void vHandlingTask( void *pvParameters )
26. {
27.     BaseType_t xEvent;
28.
29.     for ( ;; ) {
30.         /* 一直阻塞（没有时间限制，所以没有必要检测函数的返回值）
31.         这里 RTOS 的任务通知值被用作二值信号量，所以在函数退出
32.         时，任务通知值要被清 0 。要注意的是真正的应用程序不应该
33.         无限期的阻塞*/
34.         ulTaskNotifyTake( pdTRUE, /* 在退出前清 0 任务通知值 */
35.             portMAX_DELAY ); /* 无限阻塞 */
36.
37.         /* RTOS 任务通知被当作二值信号量使用
38.         当处理完所有的事情后继续等待下一个任务通知*/
39.         do {
40.             xEvent = xQueryPeripheral();
41.
42.             if ( xEvent != NO_MORE_EVENTS ) {
43.                 vProcessPeripheralEvent( xEvent );
44.             }
45.
46.         } while ( xEvent != NO_MORE_EVENTS );
47.     }
48. }

```



9.xTaskNotifyWait()

xTaskNotifyWait()函数用于实现全功能版的等待任务通知，根据用户指定的参数的不同，可以灵活的用于实现轻量级的消息队列、二值信号量、计数信号量和事件组功能，并带有超时等待。

函数原型	BaseType_t xTaskNotifyWait(uint32_t ulBitsToClearOnEntry, uint32_t ulBitsToClearOnExit, uint32_t *pulNotificationValue, TickType_t xTicksToWait);	
功能	用于等待一个任务通知，并带有超时等待。	
参数	ulBitsToClearOnEntry	ulBitsToClearOnEntry 表示在使用通知之前，将任务通知值的哪些位清 0，实现过程就是将任务的通知值与参数 ulBitsToClearOnEntry 的按位取反值按位与操作。 如果 ulBitsToClearOnEntry 设置为 0x01，那么在函数进入前，任务通知值的位 1 会被清 0，其他位保持不变。如果 ulBitsToClearOnEntry 设置为 0xFFFFFFFF (ULONG_MAX)，那么在进入函数前任务通知值的所有位都会被清 0，表示清零任务通知值。
	ulBitsToClearOnExit	ulBitsToClearOnExit 表示在函数 xTaskNotifyWait()退出前，决定任务接收到的通知值的哪些位会被清 0，实现过程就是将任务的通知值与参数 ulBitsToClearOnExit 的按位取反值按位与操作。在清 0 前，接收到的任务通知值会先被保存到形参 *pulNotificationValue 中。 如果 ulBitsToClearOnExit 设置为 0x03，那么在函数退出前，接收到的任务通知值的位 0 和位 1 会被清 0，其他位保持不变。如果 ulBitsToClearOnExit 设置为 0xFFFFFFFF (ULONG_MAX)，那么在退出函数前接收到的任务通知值的所有位都会被清 0，表示退出时清零任务通知值。
	pulNotificationValue	用于保存接收到的任务通知值。如果接收到的任务通知不需要使用，则设置为 NULL 即可。这个通知值在参数 ulBitsToClearOnExit 起作用前将通知值拷贝到 *pulNotificationValue 中。
	xTicksToWait	等待超时时间，单位为系统节拍周期。宏 pdMS_TO_TICKS 用于将单位毫秒转化为系统节拍数。
返回值	如果获取任务通知成功则返回 pdTRUE，失败则返回 pdFALSE。 CSDN @不秃也很强	

xTaskNotifyWait()函数使用实例

```
1. /* 这个任务展示使用任务通知值的位来传递不同的事件
2. 这在某些情况下可以代替事件标志组。*/
3. void vAnEventProcessingTask( void *pvParameters )
4. {
5.     uint32_t ulNotifiedValue;
6.
7.     for ( ;; ) {
8.         /* 等待任务通知，无限期阻塞（没有超时，所以没有必要检查函数返回值）
9.         这个任务的任务通知值的位由标志事件发生的任务或者中断来设置*/
10.        xTaskNotifyWait( 0x00, /* 在进入的时候不清除通知值的任何位 */
11.                        ULONG_MAX, /* 在退出的时候复位通知值为 0 */
12.                        &ulNotifiedValue, /* 任务通知值传递到变量
13.                        ulNotifiedValue 中*/
14.                        portMAX_DELAY ); /* 无限期待 */
15.
16.
17.        /* 根据任务通知值里面的各个位的值处理事情 */
18.        if ( ( ulNotifiedValue & 0x01 ) != 0 ) {
19.            /* 位 0 被置 1 */
20.            prvProcessBit0Event();
21.        }
22.
```

```

23. if ( ( ulNotifiedValue & 0x02 ) != 0 ) {
24. /* 位 1 被置 1 */
25. prvProcessBit1Event();
26. }
27.
28. if ( ( ulNotifiedValue & 0x04 ) != 0 ) {
29. /* 位 2 被置 1 */
30. prvProcessBit2Event();
31. }
32.
33. /* ... 等等 */
34. }
35. }

```



四、任务通知实验

1 任务通知代替消息队列

任务通知代替消息队列是在 FreeRTOS 中创建三个任务，其中两个任务是用于接收 任务通知，另一个任务发送任务通知。三个任务独立运行，发送消息任务是通过检测按键 的按下情况来发送消息通知，另两个任务获取消息通知，在任务通知中没有可用的通知之前就一直等待消息，一旦获取到消息通知就把消息打印在串口调试助手里。

```

1. /* FreeRTOS头文件 */
2. #include "FreeRTOS.h"
3. #include "task.h"
4. /* 开发板硬件bsp头文件 */
5. #include "bsp_led.h"
6. #include "bsp_usart.h"
7. #include "bsp_key.h"
8. #include "limits.h"
9. /***** 任务句柄 *****/
10. /*
11. * 任务句柄是一个指针，用于指向一个任务，当任务创建好之后，它就具有了一个任务句柄
12. * 以后我们要想操作这个任务都需要通过这个任务句柄，如果是自身的任务操作自己，那么
13. * 这个句柄可以为NULL。
14. */
15. static TaskHandle_t AppTaskCreate_Handle = NULL; /* 创建任务句柄 */
16. static TaskHandle_t Receive1_Task_Handle = NULL; /* Receive1_Task任务句柄 */
17. static TaskHandle_t Receive2_Task_Handle = NULL; /* Receive2_Task任务句柄 */
18. static TaskHandle_t Send_Task_Handle = NULL; /* Send_Task任务句柄 */
19.
20. /***** 内核对象句柄 *****/
21. /*
22. * 信号量，消息队列，事件标志组，软件定时器这些都属于内核的对象，要想使用这些内核
23. * 对象，必须先创建，创建成功之后会返回一个相应的句柄。实际上就是一个指针，后续我
24. * 们就可以通过这个句柄操作这些内核对象。
25. *
26. * 内核对象说白了就是一种全局的数据结构，通过这些数据结构我们可以实现任务间的通信，
27. * 任务间的事件同步等各种功能。至于这些功能的实现我们是通过调用这些内核对象的函数
28. * 来完成的
29. *
30. */
31.
32.
33. /***** 全局变量声明 *****/
34. /*
35. * 当我们在写应用程序的时候，可能需要用到一些全局变量。
36. */
37.
38.
39. /***** 宏定义 *****/
40. /*
41. * 当我们在写应用程序的时候，可能需要用到一些宏定义。
42. */
43. #define USE_CHAR 0 /* 测试字符串的时候配置为 1，测试变量配置为 0 */
44.
45. /*
46. *****/
47. * 函数声明
48. *****/
49. /*
50. static void AppTaskCreate(void); /* 用于创建任务 */
51.
52. static void Receive1_Task(void* pvParameters); /* Receive1_Task任务实现 */
53. static void Receive2_Task(void* pvParameters); /* Receive2_Task任务实现 */
54.

```

```

55. static void Send_Task(void* pvParameters);/* Send_Task任务实现 */
56.
57. static void BSP_Init(void);/* 用于初始化板载相关资源 */
58.
59. /*****
60.  * @brief 主函数
61.  * @param 无
62.  * @retval 无
63.  * @note 第一步：开发板硬件初始化
64.          第二步：创建APP应用任务
65.          第三步：启动FreeRTOS，开始多任务调度
66.  *****/
67. int main(void)
68. {
69.     BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为pdPASS */
70.
71.     /* 开发板硬件初始化 */
72.     BSP_Init();
73.     printf("这是一个FreeRTOS任务通知代替消息队列实验！\n");
74.     printf("按下KEY1或者KEY2进行任务消息通知发送\n");
75.     /* 创建AppTaskCreate任务 */
76.     xReturn = xTaskCreate((TaskFunction_t)AppTaskCreate, /* 任务入口函数 */
77.                          (const char*) "AppTaskCreate", /* 任务名字 */
78.                          (uint16_t) 512, /* 任务栈大小 */
79.                          (void*) NULL, /* 任务入口函数参数 */
80.                          (UBaseType_t) 1, /* 任务的优先级 */
81.                          (TaskHandle_t*) &AppTaskCreate_Handle);/* 任务控制块指针 */
82.     /* 启动任务调度 */
83.     if(pdPASS == xReturn)
84.         vTaskStartScheduler(); /* 启动任务，开启调度 */
85.     else
86.         return -1;
87.
88.     while(1); /* 正常不会执行到这里 */
89. }
90.
91.
92. /*****
93.  * @ 函数名 : AppTaskCreate
94.  * @ 功能说明: 为了方便管理，所有的任务创建函数都放在这个函数里面
95.  * @ 参数 : 无
96.  * @ 返回值 : 无
97.  *****/
98. static void AppTaskCreate(void)
99. {
100.     BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为pdPASS */
101.
102.     taskENTER_CRITICAL(); /* 进入临界区 */
103.
104.     /* 创建Receive1_Task任务 */
105.     xReturn = xTaskCreate((TaskFunction_t)Receive1_Task, /* 任务入口函数 */
106.                          (const char*) "Receive1_Task", /* 任务名字 */
107.                          (uint16_t) 512, /* 任务栈大小 */
108.                          (void*) NULL, /* 任务入口函数参数 */
109.                          (UBaseType_t) 2, /* 任务的优先级 */
110.                          (TaskHandle_t*) &Receive1_Task_Handle);/* 任务控制块指针 */
111.     if(pdPASS == xReturn)
112.         printf("创建Receive1_Task任务成功!\r\n");
113.
114.     /* 创建Receive2_Task任务 */
115.     xReturn = xTaskCreate((TaskFunction_t)Receive2_Task, /* 任务入口函数 */
116.                          (const char*) "Receive2_Task", /* 任务名字 */
117.                          (uint16_t) 512, /* 任务栈大小 */
118.                          (void*) NULL, /* 任务入口函数参数 */
119.                          (UBaseType_t) 3, /* 任务的优先级 */
120.                          (TaskHandle_t*) &Receive2_Task_Handle);/* 任务控制块指针 */
121.     if(pdPASS == xReturn)
122.         printf("创建Receive2_Task任务成功!\r\n");
123.
124.     /* 创建Send_Task任务 */
125.     xReturn = xTaskCreate((TaskFunction_t)Send_Task, /* 任务入口函数 */
126.                          (const char*) "Send_Task", /* 任务名字 */
127.                          (uint16_t) 512, /* 任务栈大小 */
128.                          (void*) NULL, /* 任务入口函数参数 */
129.                          (UBaseType_t) 4, /* 任务的优先级 */
130.                          (TaskHandle_t*) &Send_Task_Handle);/* 任务控制块指针 */
131.     if(pdPASS == xReturn)
132.         printf("创建Send_Task任务成功!\n\n");
133.
134.     vTaskDelete(AppTaskCreate_Handle); /* 删除AppTaskCreate任务 */
135.
136.     taskEXIT_CRITICAL(); /* 退出临界区 */
137. }

```

```

138.
139.
140.
141. /*****
142.  * @ 函数名 : Receive_Task
143.  * @ 功能说明: Receive_Task任务主体
144.  * @ 参数 :
145.  * @ 返回值 : 无
146.  *****/
147. static void Receive1_Task(void* parameter)
148. {
149.     BaseType_t xReturn = pdTRUE; /* 定义一个创建信息返回值，默认为pdPASS */
150.     #if USE_CHAR
151.         char *r_char;
152.     #else
153.         uint32_t r_num;
154.     #endif
155.     while (1)
156.     {
157.         /* BaseType_t xTaskNotifyWait(uint32_t ulBitsToClearOnEntry,
158.                                         uint32_t ulBitsToClearOnExit,
159.                                         uint32_t *pulNotificationValue,
160.                                         TickType_t xTicksToWait );
161.         * ulBitsToClearOnEntry: 当没有接收到任务通知的时候将任务通知值与此参数的取
162.         反值进行按位与运算，当此参数为0xffff或者ULONG_MAX的时候就会将任务通知值清零。
163.         * ulBits ToClearOnExit: 如果接收到了任务通知，在做完相应的处理退出函数之前将
164.         任务通知值与此参数的取反值进行按位与运算，当此参数为0xffff或者ULONG_MAX的时候
165.         就会将任务通知值清零。
166.         * pulNotification Value: 此参数用来保存任务通知值。
167.         * xTick ToWait: 阻塞时间。
168.         *
169.         * 返回值: pdTRUE: 获取到了任务通知。pdFALSE: 任务通知获取失败。
170.         */
171.         //获取任务通知，没获取到则一直等待
172.         xReturn=xTaskNotifyWait(0x0, //进入函数的时候不清除任务bit
173.                                ULONG_MAX, //退出函数的时候清除所有的bit
174.                                #if USE_CHAR
175.                                    (uint32_t *)&r_char, //保存任务通知值
176.                                #else
177.                                    &r_num, //保存任务通知值
178.                                #endif
179.                                portMAX_DELAY); //阻塞时间
180.         if( pdTRUE == xReturn )
181.         #if USE_CHAR
182.             printf("Receive1_Task 任务通知消息为 %s \n",r_char);
183.         #else
184.             printf("Receive1_Task 任务通知消息为 %d \n",r_num);
185.         #endif
186.
187.
188.         LED1_TOGGLE;
189.     }
190. }
191.
192. /*****
193.  * @ 函数名 : Receive_Task
194.  * @ 功能说明: Receive_Task任务主体
195.  * @ 参数 :
196.  * @ 返回值 : 无
197.  *****/
198. static void Receive2_Task(void* parameter)
199. {
200.     BaseType_t xReturn = pdTRUE; /* 定义一个创建信息返回值，默认为pdPASS */
201.     #if USE_CHAR
202.         char *r_char;
203.     #else
204.         uint32_t r_num;
205.     #endif
206.     while (1)
207.     {
208.         /* BaseType_t xTaskNotifyWait(uint32_t ulBitsToClearOnEntry,
209.                                         uint32_t ulBitsToClearOnExit,
210.                                         uint32_t *pulNotificationValue,
211.                                         TickType_t xTicksToWait );
212.         * ulBitsToClearOnEntry: 当没有接收到任务通知的时候将任务通知值与此参数的取
213.         反值进行按位与运算，当此参数为0xffff或者ULONG_MAX的时候就会将任务通知值清零。
214.         * ulBits ToClearOnExit: 如果接收到了任务通知，在做完相应的处理退出函数之前将
215.         任务通知值与此参数的取反值进行按位与运算，当此参数为0xffff或者ULONG_MAX的时候
216.         就会将任务通知值清零。
217.         * pulNotification Value: 此参数用来保存任务通知值。
218.         * xTick ToWait: 阻塞时间。
219.         *
220.         * 返回值: pdTRUE: 获取到了任务通知。pdFALSE: 任务通知获取失败。

```



```

221.     */
222.     //获取任务通知 ,没获取到则一直等待
223.     xReturn=xTaskNotifyWait(0x0,                //进入函数的时候不清除任务bit
224.                             ULONG_MAX,         //退出函数的时候清除所有的bit
225. #if USE_CHAR
226.                             (uint32_t *)&r_char,                //保存任务通知值
227. #else
228.                             &r_num,                //保存任务通知值
229. #endif
230.                             portMAX_DELAY);        //阻塞时间
231.     if( pdTRUE == xReturn )
232. #if USE_CHAR
233.         printf("Receive2_Task 任务通知消息为 %s \n",r_char);
234. #else
235.         printf("Receive2_Task 任务通知消息为 %d \n",r_num);
236. #endif
237.         LED2_TOGGLE;
238.     }
239. }
240.
241. /*****
242.  * @ 函数名 : Send_Task
243.  * @ 功能说明: Send_Task任务主体
244.  * @ 参数 :
245.  * @ 返回值 : 无
246.  *****/
247. static void Send_Task(void* parameter)
248. {
249.     BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为pdPASS */
250. #if USE_CHAR
251.     char test_str1[] = "this is a mail test 1";/* 邮箱消息test1 */
252.     char test_str2[] = "this is a mail test 2";/* 邮箱消息test2 */
253. #else
254.     uint32_t send1 = 1;
255.     uint32_t send2 = 2;
256. #endif
257.
258.
259.
260.     while (1)
261.     {
262.         /* KEY1 被按下 */
263.         if( Key_Scan(KEY1_GPIO_PORT,KEY1_GPIO_PIN) == KEY_ON )
264.         {
265.             /* 原型:BaseType_t xTaskNotify( TaskHandle_t xTaskToNotify,
266.                                             uint32_t ulValue,
267.                                             eNotifyAction eAction );
268.
269.             * eNoAction = 0, 通知任务而不更新其通知值。
270.             * eSetBits,      设置任务通知值中的位。
271.             * eIncrement,    增加任务的通知值。
272.             * eSetValueWithOverwrite, 覆盖当前通知
273.             * eSetValueWithoutOverwrite 不覆盖当前通知
274.             *
275.             * pdFAIL: 当参数eAction设置为eSetValueWithoutOverwrite的时候,
276.             * 如果任务通知值没有更新成功就返回pdFAIL。
277.             * pdPASS: eAction 设置为其他选项的时候统一返回pdPASS。
278.             */
279.             xReturn = xTaskNotify( Receive1_Task_Handle, /*任务句柄*/
280. #if USE_CHAR
281.                                     (uint32_t)&test_str1, /* 发送的数据，最大为4字节 */
282. #else
283.                                     send1, /* 发送的数据，最大为4字节 */
284. #endif
285.                                     eSetValueWithOverwrite );/*覆盖当前通知*/
286.             if( xReturn == pdPASS )
287.                 printf("Receive1_Task_Handle 任务通知消息发送成功!\r\n");
288.         }
289.         /* KEY2 被按下 */
290.         if( Key_Scan(KEY2_GPIO_PORT,KEY2_GPIO_PIN) == KEY_ON )
291.         {
292.             xReturn = xTaskNotify( Receive2_Task_Handle, /*任务句柄*/
293. #if USE_CHAR
294.                                     (uint32_t)&test_str2, /* 发送的数据，最大为4字节 */
295. #else
296.                                     send2, /* 发送的数据，最大为4字节 */
297. #endif
298.                                     eSetValueWithOverwrite );/*覆盖当前通知*/
299.             /* 此函数只会返回pdPASS */
300.             if( xReturn == pdPASS )
301.                 printf("Receive2_Task_Handle 任务通知消息发送成功!\r\n");
302.         }
303.         vTaskDelay(20);

```

```

304.     }
305. }
306. /*****
307.  * @ 函数名   : BSP_Init
308.  * @ 功能说明: 板级外设初始化, 所有板子上的初始化均可放在这个函数里面
309.  * @ 参数     :
310.  * @ 返回值   : 无
311.  *****/
312. static void BSP_Init(void)
313. {
314.     /*
315.      * STM32中断优先级分组为4, 即4bit都用来表示抢占优先级, 范围为: 0~15
316.      * 优先级分组只需要分组一次即可, 以后如果有其他的任务需要用到中断,
317.      * 都统一用这个优先级分组, 千万不要再分组, 切忌。
318.      */
319.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
320.
321.     /* LED 初始化 */
322.     LED_GPIO_Config();
323.
324.     /* 串口初始化 */
325.     USART_Config();
326.
327.     /* 按键初始化 */
328.     Key_GPIO_Config();
329. }
330. }
331.
332. /*****END OF FILE*****/

```



2 任务通知代替二值信号量

任务通知代替消息队列是在 FreeRTOS 中创建了三个任务, 其中两个任务是用于接收 任务通知, 另一个任务发送任务通知。三个任务独立运行, 发送通知任务是通过检测按键 的按下情况来发送通知, 另两个任务获取通知, 在任务通知中没有可用的通知之前就一直在 等待任务通知, 获取到通知以后就将通知值清 0, 这样子是为了代替二值信号量, 任务同步成功则继续执行, 然后在串口调试助手里将运行信息打印出来。

```

1.
2. /* FreeRTOS头文件 */
3. #include "FreeRTOS.h"
4. #include "task.h"
5. /* 开发板硬件bsp头文件 */
6. #include "bsp_led.h"
7. #include "bsp_usart.h"
8. #include "bsp_key.h"
9. /***** 任务句柄 *****/
10. /*
11.  * 任务句柄是一个指针, 用于指向一个任务, 当任务创建好之后, 它就具有了一个任务句柄
12.  * 以后我们要想操作这个任务都需要通过这个任务句柄, 如果是自身的任务操作自己, 那么
13.  * 这个句柄可以为NULL。
14.  */
15. static TaskHandle_t AppTaskCreate_Handle = NULL; /* 创建任务句柄 */
16. static TaskHandle_t Receive1_Task_Handle = NULL; /* Receive1_Task任务句柄 */
17. static TaskHandle_t Receive2_Task_Handle = NULL; /* Receive2_Task任务句柄 */
18. static TaskHandle_t Send_Task_Handle = NULL; /* Send_Task任务句柄 */
19.
20. /***** 内核对象句柄 *****/
21. /*
22.  * 信号量, 消息队列, 事件标志组, 软件定时器这些都属于内核的对象, 要想使用这些内核
23.  * 对象, 必须先创建, 创建成功之后会返回一个相应的句柄。实际上就是一个指针, 后续我
24.  * 们就可以通过这个句柄操作这些内核对象。
25.  *
26.  * 内核对象说白了就是一种全局的数据结构, 通过这些数据结构我们可以实现任务间的通信,
27.  * 任务间的事件同步等各种功能。至于这些功能的实现我们是通过调用这些内核对象的函数
28.  * 来完成的
29.  *
30.  */
31.
32.
33. /***** 全局变量声明 *****/
34. /*
35.  * 当我们在写应用程序的时候, 可能需要用到一些全局变量。
36.  */
37.
38.
39. /***** 宏定义 *****/
40. /*

```

```

41.  * 当我们在写应用程序的时候，可能需要用到一些宏定义。
42.  */
43.
44.
45.  /*
46.  ****
47.  *                               函数声明
48.  ****
49.  */
50.  static void AppTaskCreate(void);/* 用于创建任务 */
51.
52.  static void Receive1_Task(void* pvParameters);/* Receive1_Task任务实现 */
53.  static void Receive2_Task(void* pvParameters);/* Receive2_Task任务实现 */
54.
55.  static void Send_Task(void* pvParameters);/* Send_Task任务实现 */
56.
57.  static void BSP_Init(void);/* 用于初始化板载相关资源 */
58.
59.  /*****
60.  * @brief 主函数
61.  * @param 无
62.  * @retval 无
63.  * @note  第一步：开发板硬件初始化
64.           第二步：创建APP应用任务
65.           第三步：启动FreeRTOS，开始多任务调度
66.  *****/
67.  int main(void)
68.  {
69.      BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为pdPASS */
70.
71.      /* 开发板硬件初始化 */
72.      BSP_Init();
73.      printf("这是一个FreeRTOS任务通知代替二值信号量实验！\n");
74.      printf("按下KEY1或者KEY2进行任务与任务间的同步\n");
75.      /* 创建AppTaskCreate任务 */
76.      xReturn = xTaskCreate((TaskFunction_t )AppTaskCreate, /* 任务入口函数 */
77.                           (const char* )"AppTaskCreate",/* 任务名字 */
78.                           (uint16_t )512, /* 任务栈大小 */
79.                           (void* )NULL,/* 任务入口函数参数 */
80.                           (UBaseType_t )1, /* 任务的优先级 */
81.                           (TaskHandle_t* )&AppTaskCreate_Handle);/* 任务控制块指针 */
82.
83.      /* 启动任务调度 */
84.      if(pdPASS == xReturn)
85.          vTaskStartScheduler(); /* 启动任务，开启调度 */
86.      else
87.          return -1;
88.
89.      while(1); /* 正常不会执行到这里 */
90.  }
91.
92.  /*****
93.  * @ 函数名 : AppTaskCreate
94.  * @ 功能说明: 为了方便管理，所有的任务创建函数都放在这个函数里面
95.  * @ 参数 : 无
96.  * @ 返回值 : 无
97.  *****/
98.  static void AppTaskCreate(void)
99.  {
100.      BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为pdPASS */
101.
102.      taskENTER_CRITICAL(); /*进入临界区
103.
104.      /* 创建Receive1_Task任务 */
105.      xReturn = xTaskCreate((TaskFunction_t )Receive1_Task, /* 任务入口函数 */
106.                           (const char* )"Receive1_Task",/* 任务名字 */
107.                           (uint16_t )512, /* 任务栈大小 */
108.                           (void* )NULL, /* 任务入口函数参数 */
109.                           (UBaseType_t )2, /* 任务的优先级 */
110.                           (TaskHandle_t* )&Receive1_Task_Handle);/* 任务控制块指针 */
111.
112.      if(pdPASS == xReturn)
113.          printf("创建Receive1_Task任务成功!\r\n");
114.
115.      /* 创建Receive2_Task任务 */
116.      xReturn = xTaskCreate((TaskFunction_t )Receive2_Task, /* 任务入口函数 */
117.                           (const char* )"Receive2_Task",/* 任务名字 */
118.                           (uint16_t )512, /* 任务栈大小 */
119.                           (void* )NULL, /* 任务入口函数参数 */
120.                           (UBaseType_t )3, /* 任务的优先级 */
121.                           (TaskHandle_t* )&Receive2_Task_Handle);/* 任务控制块指针 */
122.
123.      if(pdPASS == xReturn)
124.          printf("创建Receive2_Task任务成功!\r\n");

```

```

124. /* 创建Send_Task任务 */
125. xReturn = xTaskCreate((TaskFunction_t )Send_Task, /* 任务入口函数 */
126.                      (const char* )"Send_Task",/* 任务名字 */
127.                      (uint16_t )512, /* 任务栈大小 */
128.                      (void* )NULL,/* 任务入口函数参数 */
129.                      (UBaseType_t )4, /* 任务的优先级 */
130.                      (TaskHandle_t* )&Send_Task_Handle);/* 任务控制块指针 */
131. if(pdPASS == xReturn)
132.     printf("创建Send_Task任务成功!\n\n");
133.
134. vTaskDelete(AppTaskCreate_Handle); //删除AppTaskCreate任务
135.
136. taskEXIT_CRITICAL(); //退出临界区
137. }
138.
139.
140.
141. /*****
142.  * @ 函数名 : Receive_Task
143.  * @ 功能说明: Receive_Task任务主体
144.  * @ 参数 :
145.  * @ 返回值 : 无
146.  *****/
147. static void Receive1_Task(void* parameter)
148. {
149.     while (1)
150.     {
151.         /* uint32_t ulTaskNotifyTake( BaseType_t xClearCountOnExit, TickType_t xTicksToWait );
152.          * xClearCountOnExit: pdTRUE 在退出函数的时候任务任务通知值清零, 类似二值信号量
153.          * pdFALSE 在退出函数ulTaskNotifyTake0的时候任务通知值减一, 类似计数型信号量。
154.          */
155.         //获取任务通知, 没获取到则一直等待
156.         ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
157.
158.         printf("Receive1_Task 任务通知获取成功!\n\n");
159.
160.         LED1_TOGGLE;
161.     }
162. }
163.
164. /*****
165.  * @ 函数名 : Receive_Task
166.  * @ 功能说明: Receive_Task任务主体
167.  * @ 参数 :
168.  * @ 返回值 : 无
169.  *****/
170. static void Receive2_Task(void* parameter)
171. {
172.     while (1)
173.     {
174.         /* uint32_t ulTaskNotifyTake( BaseType_t xClearCountOnExit, TickType_t xTicksToWait );
175.          * xClearCountOnExit: pdTRUE 在退出函数的时候任务任务通知值清零, 类似二值信号量
176.          * pdFALSE 在退出函数ulTaskNotifyTake0的时候任务通知值减一, 类似计数型信号量。
177.          */
178.         //获取任务通知, 没获取到则一直等待
179.         ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
180.
181.         printf("Receive2_Task 任务通知获取成功!\n\n");
182.
183.         LED2_TOGGLE;
184.     }
185. }
186.
187. /*****
188.  * @ 函数名 : Send_Task
189.  * @ 功能说明: Send_Task任务主体
190.  * @ 参数 :
191.  * @ 返回值 : 无
192.  *****/
193. static void Send_Task(void* parameter)
194. {
195.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值, 默认为pdPASS */
196.     while (1)
197.     {
198.         /* KEY1 被按下 */
199.         if( Key_Scan(KEY1_GPIO_PORT, KEY1_GPIO_PIN) == KEY_ON )
200.         {
201.             /* 原型:BaseType_t xTaskNotifyGive( TaskHandle_t xTaskToNotify ); */
202.             xReturn = xTaskNotifyGive(Receive1_Task_Handle);
203.             /* 此函数只会返回pdPASS */
204.             if( xReturn == pdTRUE )
205.                 printf("Receive1_Task_Handle 任务通知发送成功!\n\n");
206.         }

```

```

207.     /* KEY2 被按下 */
208.     if( Key_Scan(KEY2_GPIO_PORT,KEY2_GPIO_PIN) == KEY_ON )
209.     {
210.         xReturn = xTaskNotifyGive(Receive2_Task_Handle);
211.         /* 此函数只会返回pdPASS */
212.         if( xReturn == pdPASS )
213.             printf("Receive2_Task_Handle 任务通知发送成功!\r\n");
214.     }
215.     vTaskDelay(20);
216. }
217. }
218. /*****
219.  * @ 函数名 : BSP_Init
220.  * @ 功能说明: 板级外设初始化, 所有板子上的初始化均可放在这个函数里面
221.  * @ 参数 :
222.  * @ 返回值 : 无
223.  *****/
224. static void BSP_Init(void)
225. {
226.     /*
227.     * STM32中断优先级分组为4, 即4bit都用来表示抢占优先级, 范围为: 0~15
228.     * 优先级分组只需要分组一次即可, 以后如果有其他的任务需要用到中断,
229.     * 都统一用这个优先级分组, 千万不要再分组, 切忌。
230.     */
231.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
232.
233.     /* LED 初始化 */
234.     LED_GPIO_Config();
235.
236.     /* 串口初始化 */
237.     USART_Config();
238.
239.     /* 按键初始化 */
240.     Key_GPIO_Config();
241.
242. }
243.
244. /*****END OF FILE*****/

```



3 任务通知代替计数信号量

任务通知代替计数信号量是基于计数型信号量实验修改而来, 模拟停车场工作运行。并且在 FreeRTOS 中创建了两个任务: 一个是获取任务通知, 一个是发送任务通知, 两个任务独立运行, 获取通知的任务是通过按下 KEY1 按键获取, 模拟停车场停车操作, 其等待时间是 0; 发送通知的任务则是通过检测 KEY2 按键按下进行通知的发送, 模拟停车场取车操作, 并且在串口调试助手输出相应信息。

```

1.
2. /* FreeRTOS头文件 */
3. #include "FreeRTOS.h"
4. #include "task.h"
5. #include "queue.h"
6. #include "semphr.h"
7. /* 开发板硬件bsp头文件 */
8. #include "bsp_led.h"
9. #include "bsp_usart.h"
10. #include "bsp_key.h"
11. /***** 任务句柄 *****/
12. /*
13.  * 任务句柄是一个指针, 用于指向一个任务, 当任务创建好之后, 它就具有了一个任务句柄
14.  * 以后我们要想操作这个任务都需要通过这个任务句柄, 如果是自身的任务操作自己, 那么
15.  * 这个句柄可以为NULL。
16.  */
17. static TaskHandle_t AppTaskCreate_Handle = NULL; /* 创建任务句柄 */
18. static TaskHandle_t Take_Task_Handle = NULL; /* Take_Task任务句柄 */
19. static TaskHandle_t Give_Task_Handle = NULL; /* Give_Task任务句柄 */
20.
21. /***** 内核对象句柄 *****/
22. /*
23.  * 信号量, 消息队列, 事件标志组, 软件定时器这些都属于内核的对象, 要想使用这些内核
24.  * 对象, 必须先创建, 创建成功之后会返回一个相应的句柄。实际上就是一个指针, 后续我
25.  * 们就可以通过这个句柄操作这些内核对象。
26.  *
27.  * 内核对象说白了就是一种全局的数据结构, 通过这些数据结构我们可以实现任务间的通信,
28.  * 任务间的事件同步等各种功能。至于这些功能的实现我们是通过调用这些内核对象的函数
29.  * 来完成的
30.  *
31.  */

```

```

32. SemaphoreHandle_t CountSem_Handle =NULL;
33.
34. /***** 全局变量声明 *****/
35. /*
36.  * 当我们在写应用程序的时候，可能需要用到一些全局变量。
37. */
38.
39.
40. /***** 宏定义 *****/
41. /*
42.  * 当我们在写应用程序的时候，可能需要用到一些宏定义。
43. */
44.
45.
46. /*
47. *****/
48. * 函数声明
49. *****/
50. */
51. static void AppTaskCreate(void);/* 用于创建任务 */
52.
53. static void Take_Task(void* pvParameters);/* Take_Task任务实现 */
54. static void Give_Task(void* pvParameters);/* Give_Task任务实现 */
55.
56. static void BSP_Init(void);/* 用于初始化板载相关资源 */
57.
58. /*****
59.  * @brief 主函数
60.  * @param 无
61.  * @retval 无
62.  * @note 第一步：开发板硬件初始化
63.          第二步：创建APP应用任务
64.          第三步：启动FreeRTOS，开始多任务调度
65. *****/
66. int main(void)
67. {
68.     BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为pdPASS */
69.
70.     /* 开发板硬件初始化 */
71.     BSP_Init();
72.
73.     printf("这是一个任务通知代替计数信号量实验！\n");
74.     printf("车位默认值为0个，按下KEY1申请车位，按下KEY2释放车位！\n\n");
75.
76.     /* 创建AppTaskCreate任务 */
77.     xReturn = xTaskCreate((TaskFunction_t )AppTaskCreate, /* 任务入口函数 */
78.                          (const char* )"AppTaskCreate",/* 任务名字 */
79.                          (uint16_t )512, /* 任务栈大小 */
80.                          (void* )NULL,/* 任务入口函数参数 */
81.                          (UBaseType_t )1, /* 任务的优先级 */
82.                          (TaskHandle_t* )&AppTaskCreate_Handle);/* 任务控制块指针 */
83.
84.     /* 启动任务调度 */
85.     if(pdPASS == xReturn)
86.         vTaskStartScheduler(); /* 启动任务，开启调度 */
87.     else
88.         return -1;
89.
90.     while(1); /* 正常不会执行到这里 */
91. }
92.
93. /*****
94.  * @ 函数名 : AppTaskCreate
95.  * @ 功能说明: 为了方便管理，所有的任务创建函数都放在这个函数里面
96.  * @ 参数 : 无
97.  * @ 返回值 : 无
98. *****/
99. static void AppTaskCreate(void)
100. {
101.     BaseType_t xReturn = pdPASS;/* 定义一个创建信息返回值，默认为pdPASS */
102.
103.     taskENTER_CRITICAL(); //进入临界区
104.
105.     /* 创建Take_Task任务 */
106.     xReturn = xTaskCreate((TaskFunction_t )Take_Task, /* 任务入口函数 */
107.                          (const char* )"Take_Task",/* 任务名字 */
108.                          (uint16_t )512, /* 任务栈大小 */
109.                          (void* )NULL, /* 任务入口函数参数 */
110.                          (UBaseType_t )2, /* 任务的优先级 */
111.                          (TaskHandle_t* )&Take_Task_Handle);/* 任务控制块指针 */
112.
113.     if(pdPASS == xReturn)
114.         printf("创建Take_Task任务成功!\r\n");

```



```

115. /* 创建Give_Task任务 */
116. xReturn = xTaskCreate((TaskFunction_t )Give_Task, /* 任务入口函数 */
117.                      (const char* )"Give_Task",/* 任务名字 */
118.                      (uint16_t )512, /* 任务栈大小 */
119.                      (void* )NULL,/* 任务入口函数参数 */
120.                      (UBaseType_t )3, /* 任务的优先级 */
121.                      (TaskHandle_t* )&Give_Task_Handle);/* 任务控制块指针 */
122. if(pdPASS == xReturn)
123.     printf("创建Give_Task任务成功!\n\n");
124.
125. vTaskDelete(AppTaskCreate_Handle); //删除AppTaskCreate任务
126.
127. taskEXIT_CRITICAL(); //退出临界区
128. }
129.
130.
131.
132. /*****
133.  * @ 函数名 : Take_Task
134.  * @ 功能说明: Take_Task任务主体
135.  * @ 参数 :
136.  * @ 返回值 : 无
137.  *****/
138. static void Take_Task(void* parameter)
139. {
140.     uint32_t take_num = pdTRUE; /* 定义一个创建信息返回值，默认为pdPASS */
141.     /* 任务都是一个无限循环，不能返回 */
142.     while (1)
143.     {
144.         //如果KEY1被单击
145.         if( Key_Scan(KEY1_GPIO_PORT,KEY1_GPIO_PIN) == KEY_ON )
146.         {
147.             /* uint32_t ulTaskNotifyTake( BaseType_t xClearCountOnExit, TickType_t xTicksToWait );
148.              * xClearCountOnExit: pdTRUE 在退出函数的时候任务任务通知值清零，类似二值信号量
149.              * pdFALSE 在退出函数ulTaskNotifyTake0的时候任务通知值减一，类似计数型信号量。
150.              */
151.             //获取任务通知，没获取到则不等待
152.             take_num=ulTaskNotifyTake(pdFALSE, 0); //
153.             if(take_num > 0)
154.                 printf( "KEY1被按下，成功申请到停车位，当前车位为 %d \n", take_num - 1);
155.             else
156.                 printf( "KEY1被按下，车位已经没有了，请按KEY2释放车位\n" );
157.         }
158.         vTaskDelay(20); //每20ms扫描一次
159.     }
160. }
161.
162. /*****
163.  * @ 函数名 : Give_Task
164.  * @ 功能说明: Give_Task任务主体
165.  * @ 参数 :
166.  * @ 返回值 : 无
167.  *****/
168. static void Give_Task(void* parameter)
169. {
170.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
171.     /* 任务都是一个无限循环，不能返回 */
172.     while (1)
173.     {
174.         //如果KEY2被单击
175.         if( Key_Scan(KEY2_GPIO_PORT,KEY2_GPIO_PIN) == KEY_ON )
176.         {
177.             /* 原型:BaseType_t xTaskNotifyGive( TaskHandle_t xTaskToNotify ); */
178.             /* 释放一个任务通知 */
179.             xTaskNotifyGive(Take_Task_Handle); //发送任务通知
180.             /* 此函数只会返回pdPASS */
181.             if ( pdPASS == xReturn )
182.                 printf( "KEY2被按下，释放1个停车位.\n" );
183.         }
184.         vTaskDelay(20); //每20ms扫描一次
185.     }
186. }
187. /*****
188.  * @ 函数名 : BSP_Init
189.  * @ 功能说明: 板级外设初始化，所有板子上的初始化均可放在这个函数里面
190.  * @ 参数 :
191.  * @ 返回值 : 无
192.  *****/
193. static void BSP_Init(void)
194. {
195.     /*
196.      * STM32中断优先级分组为4，即4bit都用来表示抢占优先级，范围为：0~15
197.      * 优先级分组只需要分组一次即可，以后如果有其他的任务需要用到中断，

```

```

198.         * 都统一用这个优先级分组，千万不要再分组，切忌。
199.         */
200.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
201.
202.     /* LED 初始化 */
203.     LED_GPIO_Config();
204.
205.     /* 串口初始化 */
206.     USART_Config();
207.
208.     /* 按键初始化 */
209.     Key_GPIO_Config();
210.
211. }
212.
213. /*****END OF FILE*****/

```

五、实验现象

1. 任务通知代替消息队列

串口配置

端口

COM13 US1

波特率

115200

数据位

8

停止位

1

校验位

无

操作

关闭串口

这是一个FreeRTOS任务通知代替消息队列实验！

按下KEY1或者KEY2进行任务消息通知发送

创建Receive1_Task任务成功！

创建Receive2_Task任务成功！

创建Send_Task任务成功！

Receive1_Task_Handle 任务通知消息发送成功！

Receive1_Task 任务通知消息为 1

Receive1_Task_Handle 任务通知消息发送成功！

Receive1_Task 任务通知消息为 1

Receive2_Task_Handle 任务通知消息发送成功！

Receive2_Task 任务通知消息为 2

Receive2_Task_Handle 任务通知消息发送成功！

Receive2_Task 任务通知消息为 2

2 任务通知代替二值信号量

串口配置

端口

COM13 US1

波特率

115200

数据位

8

停止位

1

校验位

无

操作

关闭串口

这是一个FreeRTOS任务通知代替二值信号量实验！

按下KEY1或者KEY2进行任务与任务间的同步

创建Receive1_Task任务成功！

创建Receive2_Task任务成功！

创建Send_Task任务成功！

Receive1_Task_Handle 任务通知发送成功！

Receive1_Task 任务通知获取成功！

Receive2_Task_Handle 任务通知发送成功！

Receive2_Task 任务通知获取成功！

Receive1_Task_Handle 任务通知发送成功！

Receive1_Task 任务通知获取成功！

Receive2_Task_Handle 任务通知发送成功！

Receive2_Task 任务通知获取成功！

3 任务通知代替计数信号量

串口配置

端口	COM13 USI ▾
波特率	115200 ▾
数据位	8 ▾
停止位	1 ▾
校验位	无 ▾
操作	● 关闭串口

这是一个任务通知代替计数信号量实验!
车位默认值为0个, 按下KEY1申请车位, 按下KEY2释放车位!

创建Take_Task任务成功!

创建Give_Task任务成功!

KEY1被按下, 车位已经没有了, 请按KEY2释放车位

KEY1被按下, 车位已经没有了, 请按KEY2释放车位

KEY2被按下, 释放1个停车位。

KEY2被按下, 释放1个停车位。

KEY1被按下, 成功申请到停车位, 当前车位为 1

KEY1被按下, 成功申请到停车位, 当前车位为 0

KEY1被按下, 车位已经没有了, 请按KEY2释放车位

KEY1被按下, 车位已经没有了, 请按KEY2释放车位

KEY2被按下, 释放1个停车位。

KEY2被按下, 释放1个停车位。

KEY2被按下, 释放1个停车位。CSDN @不秃也很强